Redesigning `heft_train_curriculum.ipynb`: revised plan

For approval before the notebook is changed
Supersedes `docs/curriculum_plan.pdf` (October 29, 2026)

October 29, 2026

Abstract

This revised plan expands the original three curriculum fixes to include two additional changes that were decided after the initial plan was written. First, the custom loss functions (`CHI2` and the custom `L1Loss`) are removed in favour of PyTorch's built-in `nn.L1Loss`, which has proved more stable and gives better results. Second, `noise_var` is removed as a training target: because the data obey Poisson statistics with a known scale, the variance is analytically determined by the cross section itself and need not be learned. Together with the three curriculum fixes — corrected sort direction, a new `DIFFICULTY` switch, and a new `PACING` switch — the notebook is updated in five targeted places. No cell is modified until you approve.

1 Changes 1–2: loss function and training target

1.1 Revert to `nn.L1Loss`

The current notebook offers two loss options, selected by a `LOSS` switch: a custom `CHI2` (Pearson χ^2) loss and a custom `L1Loss` with a noise floor. Testing showed that results with the custom `L1Loss` are worse than with the earlier, simpler approach. The fix is to remove both custom loss classes and replace them with PyTorch's built-in `nn.L1Loss`, which computes the mean absolute error (MAE) between the model's prediction $\hat{f}_k$ and the true bin cross section σ_k:

$$\mathcal{L} = \frac{1}{K} \sum_{k=1}^{K} |\hat{f}_k - \sigma_k|.$$

This treats every bin equally, with no weighting by cross section or noise estimate. It is numerically stable and requires no auxiliary inputs.

The `LOSS` switch is retained in the switches cell for documentary clarity, but it now simply instantiates `nn.L1Loss()` with no branching. The custom loss class definitions are removed entirely. A comment in the switches cell explains why a custom loss was tried and why the built-in is preferred.

1.2 Remove `noise_var` from the training target

The target array currently has two columns,

$$Y = \begin{bmatrix} \sigma_k & \hat{v}_k \end{bmatrix},$$

where $\hat{v}_k$ is a noise-variance estimate included for use in the CHI2 loss denominator. With that loss gone, the second column serves no purpose in training. The new target array is simply

$$Y = [\sigma_k] .$$

Why `noise_var` is not needed as a target

The data are scaled Poisson counts. Each spectrum j consists of $N_{\text{ev}} = 40\,000$ events with equal weight w_j; bin k receives n_k events, so

$$\sigma_k = w_j\, n_k, \qquad \text{Var}(\sigma_k) = w_j^2\, n_k = w_j\, \sigma_k.$$

The variance is therefore determined exactly by the cross section itself, up to the per-spectrum weight w_j. There is no free parameter to learn: any time a variance estimate is needed (for diagnostics, for a future weighted loss, or for uncertainty propagation), it can be computed analytically as $w_j\, \hat{\sigma}_k$ from the model's own prediction $\hat{\sigma}_k$.

The `noise_var` column is *kept in the dataframe*; removing it from training does not require removing it from the data file. This preserves it for diagnostic use.

Cells that change. Two cells are affected by these two changes.

1. *Switches cell:* `LOSS` is set to `'L1'` and the custom loss class definitions are removed. A one-line comment explains what was tried and why the built-in is used instead.
2. *Y-array construction cell:* the line

   ```
   Y = df[[NN.TARGET, 'noise_var']].to_numpy(dtype=np.float32)
   ```

   becomes

   ```
   Y = df[[NN.TARGET]].to_numpy(dtype=np.float32)
   ```

   and the training loop is updated to pass `y.squeeze()` (or `y[:, 0]`) to the loss function rather than unpacking two columns.

2 Changes 3–5: the curriculum

These three changes are unchanged from the original plan. They are reproduced here for completeness.

Background: easy means large cross section

Bengio et al. (2009) showed that training on examples ordered from easy to hard often converges faster and to a better minimum. For HEFTNET, a training row is one bin (x_k, θ, σ_k). The SNR of that bin is

$$\text{SNR}_k = \sqrt{\frac{\sigma_k}{w_j}},$$

so a large σ_k means a reliable gradient and a small σ_k means a noisy one. **Easy rows are those with the largest cross section, not the smallest.**

2.1 Fix the sort direction (Change 3)

Cell 19 currently sorts in ascending order:

```
order = np.argsort(key, kind='stable')
```

This puts the hardest (smallest) examples first. The fix is:

```
order = np.argsort(score, kind='stable')[::-1]
```

The variable is renamed `score` for clarity, and a comment explains why descending order is correct.

2.2 Replace `CURRICULUM_BY` with `DIFFICULTY` (Change 4)

The existing switch conflates the difficulty criterion (what makes an example easy) with the level at which it is applied (per-bin or per-spectrum). These are split into a single, clearly named switch:

`DIFFICULTY` value	sort key	granularity
`'bin_sigma'`	σ_k of that bin	per bin
`'spectrum_sigma'`	$\sigma_{\text{tot}} = \sum_k \sigma_k$ of the parent spectrum	per spectrum
`'spectrum_snr'`	$\sigma_{\text{tot}} / \sqrt{\overline{w_j}}$	per spectrum

The `'spectrum_snr'` option requires the per-spectrum event weight w_j. This is not in the current training file; the option is included in the switch for future use but raises `NotImplementedError` if selected, with a comment explaining what to add.

2.3 Add `PACING` switch and log ramp (Change 5)

A new `PACING` switch controls the shape of the fraction ramp:

- `'linear'`: $f(i) = \min\big(1,\ f_0 + (1 - f_0)\, i / i_{\text{ramp}}\big)$. This is the current behaviour.
- `'log'`: $f(i) = \min\big(1,\ f_0^{\,1 - i/i_{\text{ramp}}}\big)$. At $i = 0$ this gives f_0; at $i = i_{\text{ramp}}$ it gives 1. The growth is slow at first and accelerates, spending more iterations on the easiest examples before the ramp opens up.

The `CurriculumLoader.fraction` method is updated with an `if/else` block for the two cases, and the constructor takes `pacing` as an argument. The docstring explains both shapes and why the sort order matters.

The `DIRNAME` string is updated from `_curriculum_bin` / `_curriculum_spectrum` to `_difficulty_<value>_pacin` so that run folders are self-describing.

3 Summary of all five changes

#	cell	what changes	replaces	status
1	Switches	remove custom loss classes; use `nn.L1Loss`	custom `CHI2` / `L1Loss`	new
2	Y-array	drop `'noise_var'` column from `Y`	two-column target	new
3	Sorting	add `[::-1]` for descending order	ascending sort	from v1
4	Switches	`DIFFICULTY` replaces `CURRICULUM_BY`	`CURRICULUM_BY`	from v1
5	`CurriculumLoader`	add `PACING` and log ramp	linear only	from v1

4 Experiments to run after the changes

Four short runs (3×20k iterations, seeded, `nn.L1Loss`):

run	curriculum	`DIFFICULTY`	`PACING`
baseline	no	—	—
A	yes	`'bin_sigma'`	`'linear'`
B	yes	`'bin_sigma'`	`'log'`
C	yes	`'spectrum_sigma'`	`'log'`

Three seeds per run give enough spread to assess significance. The baseline run also uses `nn.L1Loss`, so the comparison is clean. Once a best curriculum setting is identified, a full 200k-iteration run follows.

Previous curriculum results are no longer comparable

The curriculum runs recorded in the report used the wrong sort direction *and* a custom loss function. They are not a valid baseline for the new experiments. All comparisons should be made against the freshly seeded `nn.L1Loss` baseline above.

5 What counts as success

A curriculum is worth keeping if, compared to the plain baseline with the same seeds and same number of iterations, it achieves a statistically meaningful improvement on the 1 000 holdout spectra: a lower median χ^2/N (evaluated *offline*, not used for training) by more than the run-to-run spread, and/or a lower fraction of spectra with $\chi^2/N > 2$. If neither criterion is met, the plain `nn.L1Loss` training is the default.

6 Changes not made by this plan

- `DIFFICULTY = 'spectrum_snr'` is stubbed out (`NotImplementedError`); it needs the per-spectrum event weight w_j added to the training file.
- Seeding (`torch.manual_seed` and `numpy.random.seed`) is a separate one-cell change, to be applied to both `heft_train.ipynb` and `heft_train_curriculum.ipynb` together.
- Nothing changes in `heft_test.ipynb`, `heft_checks.ipynb`, or `heft_export.ipynb`.
- The `noise_var` column is kept in the dataframe and data files; only its inclusion in the training target array is removed.

Decisions for you before I proceed

1. **Loss function.** Confirmed: use `nn.L1Loss` and remove the custom loss classes. Any objection?
2. **Noise-var target.** Confirmed: remove `'noise_var'` from `Y`. Keep the column in the dataframe. Any objection?
3. **Sort direction.** Easy = large cross section = descending sort. Any objection?

4. **Difficulty options.** `'bin_sigma'`, `'spectrum_sigma'`, `'spectrum_snr'` (stubbed). Any additions or changes?
5. **DIRNAME naming.** `runs/L1Loss_difficulty_bin_sigma_pacing_log`. Acceptable, or should we abbreviate?
6. **Seeding.** Same time as these changes, or separately afterwards?